Instituto Politécnico Formosa

Tecnicatura Superior en Desarrollo de Software Multiplataforma

TRABAJO PRÁCTICO N.º 1

Análisis y diagnóstico de fragmentos de código

Espacio curricular	Taller Complementario «React Native»
Docentes	ROLÓN Lautaro Emanuel MERELES Juliana Aracelli
Ciclo lectivo	2026 — Segundo cuatrimestre
Modalidad	Individual
Fecha de entrega	07 / 08 / 2026

1. Condiciones de entrega

- El trabajo es **individual**. Cada estudiante debe presentar su propia resolución.
- La entrega se realiza únicamente a través del **aula virtual (Google Classroom)**, en la tarea correspondiente al Trabajo Práctico N.º 1.
- Formato de entrega: un único documento en **PDF**. Nombre de archivo sugerido: `TP1_Apellido_Nombre.pdf`.
- El PDF debe incluir carátula con tus datos y las respuestas numeradas según cada ejercicio. No hace falta copiar el código completo: alcanza con indicar el número de ejercicio y las líneas a las que hacés referencia.
- Fecha límite: la publicada en el aula virtual. **No se aceptarán entregas fuera de término** ni por otros medios (correo, mensajes, etc.).
- Las respuestas deben estar redactadas **con tus propias palabras**. Respuestas idénticas entre estudiantes o copiadas textualmente de internet se considerarán no válidas.

2. Consignas generales

En la **Parte A** vas a encontrar nueve fragmentos de código de React Native. Cada uno pertenece a una pequeña aplicación desarrollada con Expo. Para *cada* fragmento respondé:

a) ¿La aplicación funcionaría con este código? Elegí una opción: *funciona correctamente* / *funciona, pero con advertencias (warnings) o problemas ocultos* / *no funciona (se produce un error)*.

b) Si detectás un problema, explicá cuál es e indicá la línea o líneas donde se encuentra.

c) Clasificá el problema: **error de ejecución**, **advertencia (warning)**, **problema de rendimiento u optimización** o **mala práctica**.

d) Fundamentá con tus propias palabras por qué sucede y proponé cómo lo corregirías.

> **Importante:** no todos los fragmentos contienen errores. Alguno puede estar perfectamente escrito; parte del trabajo consiste, también, en poder justificar por qué un código está bien.

3. Cómo verificar los fragmentos (recomendado)

Podés —y es muy recomendable— copiar cada fragmento en un **proyecto de ejemplo** y ejecutarlo de forma local para verificar su comportamiento real: pantallas de error, advertencias en la consola, demoras al desplazarse, etc.

1. Creá un proyecto de prueba con Expo y abrilo en tu editor.
2. Reemplazá el contenido de `App.js` por el fragmento que quieras analizar.
3. Iniciá el proyecto y probalo en Expo Go o en un emulador.
4. Prestá atención a la terminal y a la consola: muchos problemas se manifiestan ahí y no en la pantalla.

```
Terminal
# Crear el proyecto de prueba (plantilla JavaScript)
$ npx create-expo-app@latest tp1-pruebas --template blank
$ cd tp1-pruebas
$ npx expo start
```

Los ejercicios que consumen datos usan **JSONPlaceholder** (https://jsonplaceholder.typicode.com), una API pública y gratuita de prueba, así que podés ejecutarlos sin configurar nada adicional.

4. Parte A — Análisis de fragmentos de código

Respondé los puntos **a)** a **d)** de las consignas generales para cada uno de los siguientes ejercicios. Están ordenados, aproximadamente, de menor a mayor complejidad.

Ejercicio 1

El siguiente componente debería mostrar un mensaje de bienvenida en la pantalla principal de la aplicación.

App.js

```js
import { View } from 'react-native';

export default function App() {
  return (
    <View>
      Bienvenido al Taller de React Native
    </View>
  );
}
```

Ejercicio 2

Un contador que debería sumar 1 cada vez que el usuario toca el botón.

Contador.js

```js
import { useState } from 'react';
import { View, Text, Button } from 'react-native';

export default function Contador() {
  const [contador, setContador] = useState(0);

  const incrementar = () => {
    contador = contador + 1;
  };

  return (
    <View>
      <Text>Toques: {contador}</Text>
      <Button title="Sumar" onPress={incrementar} />
    </View>
  );
}
```

Ejercicio 3

Se quiere mostrar en pantalla una lista fija de materias de la carrera.

ListaMaterias.js

```
import { View, Text } from 'react-native';

const materias = ['Programación', 'Base de Datos', 'Inglés Técnico'];

export default function ListaMaterias() {
  return (
    <View>
      {materias.map((materia) => (
        <Text>{materia}</Text>
      ))}
    </View>
  );
}
```

Además: ¿qué función cumple la prop `key` cuando renderizamos listas de elementos?

Ejercicio 4

Al abrir esta pantalla, la app debería descargar los datos de un usuario desde una API y mostrar su nombre.

Perfil.js

```
import { useState, useEffect } from 'react';
import { View, Text } from 'react-native';

export default function Perfil() {
  const [usuario, setUsuario] = useState(null);

  useEffect(() => {
    fetch('https://jsonplaceholder.typicode.com/users/1')
      .then((res) => res.json())
      .then((datos) => setUsuario(datos));
  });

  return (
    <View>
      <Text>Usuario: {usuario ? usuario.name : 'Cargando...'}</Text>
    </View>
  );
}
```

Ejercicio 5

Se descargan publicaciones desde una API y se muestran en una lista desplazable.

ListaPosts.js

```js
import { useState, useEffect } from 'react';
import { FlatList, Text } from 'react-native';

export default function ListaPosts() {
  const [posts, setPosts] = useState([]);

  useEffect(async () => {
    const res = await fetch('https://jsonplaceholder.typicode.com/posts');
    const datos = await res.json();
    setPosts(datos);
  }, []);

  return (
    <FlatList
      data={posts}
      keyExtractor={(item) => String(item.id)}
      renderItem={({ item }) => <Text>{item.title}</Text>}
    />
  );
}
```

Ejercicio 6

Este componente recibe por props el historial de movimientos de una cuenta, con más de 5.000 elementos.

Historial.js

```js
import { ScrollView, Text } from 'react-native';

export default function Historial({ movimientos }) {
  // "movimientos" llega por props con más de 5.000 elementos
  return (
    <ScrollView>
      {movimientos.map((mov) => (
        <Text key={mov.id}>
          {mov.fecha} - {mov.detalle}
        </Text>
      ))}
    </ScrollView>
  );
}
```

Además: si propusieras un componente alternativo, ¿cuál sería y qué ventaja concreta aporta en este caso?

Ejercicio 7

Un cronómetro simple que cuenta los segundos desde que se abre la pantalla. Suponé que, mientras se usa la app, esta pantalla se abre y se cierra muchas veces.

Cronometro.js

```
import { useState, useEffect } from 'react';
import { View, Text } from 'react-native';

export default function Cronometro() {
  const [segundos, setSegundos] = useState(0);

  useEffect(() => {
    setInterval(() => {
      setSegundos((s) => s + 1);
    }, 1000);
  }, []);

  return (
    <View>
      <Text>Segundos transcurridos: {segundos}</Text>
    </View>
  );
}
```

Pista: ¿qué debería devolver la función que se le pasa a `useEffect` en casos como este?

Ejercicio 8

Un formulario de registro: el usuario escribe su nombre en el campo de texto y, si algo falla, puede tocar el botón «Reintentar».

Registro.js

```
import { useState } from 'react';
import { View, Text, TextInput, Button } from 'react-native';

export default function Registro() {
  const [intentos, setIntentos] = useState(0);

  function CampoNombre() {
    const [nombre, setNombre] = useState('');
    return (
      <TextInput
        value={nombre}
        onChangeText={setNombre}
        placeholder="Escribí tu nombre"
      />
    );
  }

  return (
    <View>
      <CampoNombre />
      <Text>Intentos: {intentos}</Text>
      <Button title="Reintentar" onPress={() => setIntentos(intentos + 1)} />
    </View>
  );
}
```

Pista: probalo en tu proyecto. Escribí un nombre en el campo de texto y después tocá el botón. ¿Qué pasa con lo que habías escrito? ¿Por qué?

Ejercicio 9

Un selector de tema claro/oscuro para la aplicación.

SelectorTema.js

```
import { useState } from 'react';
import { View, Text, Pressable, StyleSheet } from 'react-native';

export default function SelectorTema() {
  const [oscuro, setOscuro] = useState(false);

  return (
    <View style={[styles.pantalla, oscuro && styles.pantallaOscura]}>
      <Text style={oscuro ? styles.textoClaro : styles.textoBase}>
        Modo {oscuro ? 'oscuro' : 'claro'} activado
      </Text>
      <Pressable style={styles.boton} onPress={() => setOscuro(!oscuro)}>
        <Text style={styles.textoBoton}>Cambiar tema</Text>
      </Pressable>
    </View>
  );
}

const styles = StyleSheet.create({
  pantalla: { flex: 1, alignItems: 'center', justifyContent: 'center' },
  pantallaOscura: { backgroundColor: '#1a1a2e' },
  textoBase: { color: '#111', fontSize: 18 },
  textoClaro: { color: '#fff', fontSize: 18 },
  boton: { marginTop: 16, padding: 12, backgroundColor: '#4361ee' },
  textoBoton: { color: '#fff', fontWeight: 'bold' },
});
```

5. Parte B — Preguntas de desarrollo

Respondé con tus palabras. Se valora más una explicación conceptual clara que una definición memorizada.

1. Sobre el array de dependencias de `useEffect`: explicá qué diferencia hay entre estas tres formas y cuándo se ejecuta el efecto en cada caso: `useEffect(fn)` — sin segundo argumento; `useEffect(fn, [])` — con array vacío; `useEffect(fn, [valor])` — con dependencias.
2. ¿Qué hace que un componente de React Native se vuelva a renderizar? ¿Qué relación tiene esto con `useState`?
3. ¿Qué ventajas tiene definir los estilos con `StyleSheet.create()` en lugar de escribir los objetos de estilo en línea, directamente dentro del JSX?
4. ¿Qué rol cumple **Expo** dentro del ecosistema de React Native? Mencioná al menos dos ventajas de usarlo y una limitación o situación en la que pueda quedarse corto.

6. Criterios de evaluación

Criterio	Puntaje
Parte A — Identificación y diagnóstico correcto de cada fragmento	30 pts
Parte A — Clasificación del tipo de problema	15 pts
Parte A — Fundamentación con palabras propias y corrección propuesta	25 pts
Parte B — Preguntas de desarrollo	20 pts
Presentación, prolijidad y entrega en tiempo y forma	10 pts
Total	**100 pts**

¡Éxitos! Ante cualquier duda, consultá en clase o a través del aula virtual.